Sorting

Content notes, classwork and homework with worked C++ solutions.

Lesson 1: Sorting Basics I

Quick recap: Bubble sort (and early exit) • Selection sort • Swap and comparison counting

Theory & Nuances

What does sorting mean?

Sorting puts the elements of an array in order, usually smallest to largest (ascending). Sorted data is much easier to work with: the minimum and maximum sit at the two ends, duplicates sit next to each other, and you can search it fast with binary search. Every simple sorting algorithm is built from just two operations: COMPARE two elements, and SWAP two elements that are in the wrong order.

Bubble sort

Walk through the array comparing each pair of neighbours arr[j] and arr[j + 1], and swap them if the left one is bigger. After the first full pass the largest element has "bubbled" to the last position, which is now final. Each next pass can stop one position earlier, so pass p only checks j from 0 to n - p - 1. After n - 1 passes the array is sorted.

Early exit

If a whole pass makes no swaps, every neighbour pair is already in order, so the array is sorted and we can stop. Keep a bool swapped = false; before each pass, set it to true on every swap, and break if it is still false at the end of the pass. With this check, an already sorted array needs just one pass: O(n) in the best case instead of O(n^2).

Selection sort

Selection sort builds the sorted part from the left. In pass i, scan the unsorted part arr[i..n-1], find the INDEX of the smallest element, and swap it into position i. Now arr[0..i] is sorted and final. It always makes about n^2 / 2 comparisons, even on sorted input (no early exit), but at most n - 1 swaps - useful when swapping is expensive.

Counting swaps and comparisons

Counting operations is how we compare algorithms. Add a counter next to each comparison and each swap. Bubble sort needs at most n(n - 1) / 2 comparisons and exactly one swap for every pair that is out of order (an inversion): {4, 3, 2, 1} has 6 inversions, so 6 swaps. Selection sort does n(n - 1) / 2 comparisons every time but only up to n - 1 swaps. Both are O(n^2) in the worst case.

Key Points / Common Mistakes

- Bubble sort inner loop is j < n - 1 - i (or n - pass): comparing arr[j] with arr[j + 1] at j = n - 1 reads past the end.
- Reset swapped = false at the START of every pass, not once before all passes.
- Selection sort remembers the INDEX of the minimum, not just its value - you need the index to swap.
- Swap only once per selection pass, after the scan, not every time a smaller value is seen.
- Change > to < in the comparison to sort in descending order; nothing else changes.
- Bubble swaps = number of inversions; selection swaps <= n - 1.

Classwork Problems

1. Bubble sort implementation

```
#include <iostream>
#include <vector>
using namespace std;

void printArray(const vector<int> &arr) {
    for (int x : arr) {
        cout << x << " ";
    }
    cout << endl;
}

void bubbleSort(vector<int> &arr) {
    int n = arr.size();
    for (int pass = 1; pass < n; pass++) {
        bool swapped = false;
        for (int j = 0; j < n - pass; j++) {     // last pass-1 items are final
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        cout << "  after pass " << pass << ": ";
        printArray(arr);
        if (!swapped) {                          // early exit
            cout << "  no swaps in this pass - already sorted, stop" << endl;
            break;
        }
    }
}

int main() {
    vector<int> a = {5, 1, 4, 2, 8};
    cout << "Sorting: ";
    printArray(a);
    bubbleSort(a);

    vector<int> b = {1, 2, 3, 4, 5};
    cout << "Sorting: ";
    printArray(b);
    bubbleSort(b);
    return 0;
}
```

Output

```
Sorting: 5 1 4 2 8
  after pass 1: 1 4 2 5 8
  after pass 2: 1 2 4 5 8
  after pass 3: 1 2 4 5 8
  no swaps in this pass - already sorted, stop
Sorting: 1 2 3 4 5
  after pass 1: 1 2 3 4 5
  no swaps in this pass - already sorted, stop
```

2. Selection sort implementation

```
#include <iostream>
#include <vector>
using namespace std;

void printArray(const vector<int> &arr) {
    for (int x : arr) {
        cout << x << " ";
    }
    cout << endl;
}

void selectionSort(vector<int> &arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {        // scan the unsorted part
            if (arr[j] < arr[minIdx]) {
                minIdx = j;
            }
        }
        swap(arr[i], arr[minIdx]);               // one swap per pass
        cout << "pass " << i + 1 << ": min = " << arr[i] << " -> ";
        printArray(arr);
    }
}

int main() {
    vector<int> arr = {64, 25, 12, 22, 11};
    cout << "Before: ";
    printArray(arr);
    selectionSort(arr);
    cout << "After : ";
    printArray(arr);
    return 0;
}
```

Output

```
Before: 64 25 12 22 11
pass 1: min = 11 -> 11 25 12 22 64
pass 2: min = 12 -> 11 12 25 22 64
pass 3: min = 22 -> 11 12 22 25 64
pass 4: min = 25 -> 11 12 22 25 64
After : 11 12 22 25 64
```

3. Trace selection sort on paper

```
#include <iostream>
#include <vector>
using namespace std;

// prints the sorted part, a bar, then the unsorted part
void showSplit(const vector<int> &arr, int sortedCount) {
    cout << "       ";
    for (int k = 0; k < (int)arr.size(); k++) {
        if (k == sortedCount) {
            cout << "| ";
        }
        cout << arr[k] << " ";
    }
    cout << endl;
}

int main() {
    vector<int> arr = {29, 10, 14, 37, 13};
    int n = arr.size();
    cout << "Start:" << endl;
    showSplit(arr, 0);

    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx]) {
                minIdx = j;
            }
        }
        cout << "i = " << i << ": smallest in the unsorted part is " << arr[minIdx]
             << " (index " << minIdx << "), swap with index " << i << endl;
        swap(arr[i], arr[minIdx]);
        showSplit(arr, i + 1);
    }
    return 0;
}
```

Output

```
Start:
       | 29 10 14 37 13
i = 0: smallest in the unsorted part is 10 (index 1), swap with index 0
       10 | 29 14 37 13
i = 1: smallest in the unsorted part is 13 (index 4), swap with index 1
       10 13 | 14 37 29
i = 2: smallest in the unsorted part is 14 (index 2), swap with index 2
       10 13 14 | 37 29
i = 3: smallest in the unsorted part is 29 (index 4), swap with index 3
       10 13 14 29 | 37
```

Homework Problems

1. Count total swaps in bubble sort

```
#include <iostream>
#include <vector>
using namespace std;

void countBubble(vector<int> arr) {             // a copy: the caller's array is untouched
    int n = arr.size();
    int swaps = 0, comparisons = 0;
    for (int pass = 1; pass < n; pass++) {
        bool swapped = false;
        for (int j = 0; j < n - pass; j++) {
            comparisons++;
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swaps++;
                swapped = true;
            }
        }
        if (!swapped) {
            break;
        }
    }
    cout << "comparisons = " << comparisons << ", swaps = " << swaps << endl;
}

int main() {
    vector<int> a = {5, 1, 4, 2, 8};
    vector<int> b = {4, 3, 2, 1};
    vector<int> c = {1, 2, 3, 4};

    cout << "{5, 1, 4, 2, 8}: ";
    countBubble(a);
    cout << "{4, 3, 2, 1}   : ";
    countBubble(b);
    cout << "{1, 2, 3, 4}   : ";
    countBubble(c);
    return 0;
}
```

Output

```
{5, 1, 4, 2, 8}: comparisons = 9, swaps = 4
{4, 3, 2, 1}   : comparisons = 6, swaps = 6
{1, 2, 3, 4}   : comparisons = 3, swaps = 0
```

2. Sort an array in descending order

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> arr = {3, 9, 1, 7, 5};
    int n = arr.size();

    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] < arr[j + 1]) {          // < instead of > : bigger goes first
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) {
            break;
        }
    }

    cout << "Descending: ";
    for (int x : arr) {
        cout << x << " ";
    }
    cout << endl;
    return 0;
}
```

Output

```
Descending: 9 7 5 3 1
```

Lesson 2: Sorting Basics II

Quick recap: Insertion sort • Stability of sorting algorithms • Time complexity comparison (O(N^2))

Theory & Nuances

Insertion sort

Insertion sort works like sorting playing cards in your hand. The left part arr[0..i-1] is always sorted. Take the next element key = arr[i], and while the element to its left is bigger, SHIFT that element one step right. When you reach a smaller-or-equal element (or the start), drop key into the gap. Shifting is cheaper than swapping: one assignment instead of three.

Why insertion sort is good on nearly sorted data

Each element only moves left past the elements that are bigger than it, so the total number of shifts equals the number of inversions. A sorted array needs 0 shifts and just n - 1 comparisons: O(n). A reversed array needs the most shifts: n(n - 1) / 2, which is O(n^2). That is why real libraries use insertion sort for small or almost sorted pieces.

Stability

A sort is STABLE if elements with equal keys keep their original relative order. Example: sorting students by marks, Asha (85) appears before Ravi (85) in the input; a stable sort keeps Asha before Ravi. This matters when you sort by one thing after another (first by name, then by marks). Bubble sort and insertion sort are stable because they only move an element past a STRICTLY bigger one. Selection sort is NOT stable: its long swap can jump an element over an equal one.

Time complexity comparison

Bubble, selection and insertion sort are all O(n^2) in the worst and average case, so doubling n makes them about 4 times slower. Best cases differ: bubble (with early exit) and insertion are O(n) on sorted input, selection is always O(n^2). All three sort in place with O(1) extra memory. For large n (say 10^5) O(n^2) is far too slow; that is what std::sort and its O(n log n) are for.

Key Points / Common Mistakes

- Save arr[i] in key BEFORE shifting - the shifts overwrite arr[i].
- The inner loop needs j >= 0 checked FIRST: while (j >= 0 && arr[j] > key).
- Use > (not >=) when shifting, or equal elements get reordered and the sort is no longer stable.
- After the loop the gap is at j + 1, not j.
- Stable: bubble, insertion. Not stable: selection.
- Best case O(n): bubble (with early exit) and insertion. Selection is O(n^2) even when sorted.

Classwork Problems

1. Insertion sort implementation

```
#include <iostream>
#include <vector>
using namespace std;

void printArray(const vector<int> &arr) {
    for (int x : arr) {
        cout << x << " ";
    }
    cout << endl;
}

void insertionSort(vector<int> &arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];                       // the card we pick up
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {        // shift bigger ones right
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;                       // drop it into the gap
        cout << "insert " << key << ": ";
        printArray(arr);
    }
}

int main() {
    vector<int> arr = {12, 11, 13, 5, 6};
    cout << "Before   : ";
    printArray(arr);
    insertionSort(arr);
    cout << "Sorted   : ";
    printArray(arr);
    return 0;
}
```

Output

```
Before   : 12 11 13 5 6
insert 11: 11 12 13 5 6
insert 13: 11 12 13 5 6
insert 5: 5 11 12 13 6
insert 6: 5 6 11 12 13
Sorted   : 5 6 11 12 13
```

2. Trace insertion sort shifts

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> arr = {5, 2, 4, 6, 1, 3};
    int n = arr.size();
    int totalShifts = 0;

    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        int shifts = 0;
        cout << "key = " << key << ":";
        while (j >= 0 && arr[j] > key) {
            cout << " shift " << arr[j];
            arr[j + 1] = arr[j];
            j--;
            shifts++;
        }
        arr[j + 1] = key;
        totalShifts += shifts;
        if (shifts == 0) {
            cout << " no shift";
        }
        cout << "  -> placed at index " << j + 1 << endl;
    }

    cout << "Sorted: ";
    for (int x : arr) {
        cout << x << " ";
    }
    cout << endl;
    cout << "Total shifts = " << totalShifts << endl;
    return 0;
}
```

Output

```
key = 2: shift 5  -> placed at index 0
key = 4: shift 5  -> placed at index 1
key = 6: no shift  -> placed at index 3
key = 1: shift 6 shift 5 shift 4 shift 2  -> placed at index 0
key = 3: shift 6 shift 5 shift 4  -> placed at index 2
Sorted: 1 2 3 4 5 6
Total shifts = 9
```

3. Compare stability of Bubble vs Selection

```
#include <iostream>
#include <string>
using namespace std;

// each item has a key (used for sorting) and a tag (to see the original order)
void show(string name, int key[], char tag[], int n) {
    cout << name;
    for (int i = 0; i < n; i++) {
        cout << key[i] << tag[i] << " ";
    }
    cout << endl;
}

int main() {
    int n = 3;
    int k1[] = {4, 4, 1};
    char t1[] = {'a', 'b', 'c'};
    int k2[] = {4, 4, 1};
    char t2[] = {'a', 'b', 'c'};
    show("Input     : ", k1, t1, n);

    // bubble sort: only neighbours swap, and only if strictly bigger
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (k1[j] > k1[j + 1]) {
                swap(k1[j], k1[j + 1]);
                swap(t1[j], t1[j + 1]);
            }
        }
    }
    show("Bubble    : ", k1, t1, n);

    // selection sort: a long-distance swap can jump over an equal key
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (k2[j] < k2[minIdx]) {
                minIdx = j;
            }
        }
        swap(k2[i], k2[minIdx]);
        swap(t2[i], t2[minIdx]);
    }
    show("Selection : ", k2, t2, n);

    cout << "Bubble keeps 4a before 4b (stable)." << endl;
    cout << "Selection puts 4b before 4a (not stable)." << endl;
    return 0;
}
```

Output

```
Input     : 4a 4b 1c
Bubble    : 1c 4a 4b
Selection : 1c 4b 4a
Bubble keeps 4a before 4b (stable).
Selection puts 4b before 4a (not stable).
```

Homework Problems

1. Sort an array of strings by first letter

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

int main() {
    vector<string> words = {"mango", "apple", "melon", "banana", "avocado", "cherry", "blueberry"};
    int n = words.size();

    // insertion sort comparing only the first letter
    for (int i = 1; i < n; i++) {
        string key = words[i];
        int j = i - 1;
        while (j >= 0 && words[j][0] > key[0]) {
            words[j + 1] = words[j];
            j--;
        }
        words[j + 1] = key;
    }

    for (const string &w : words) {
        cout << w << " ";
    }
    cout << endl;
    cout << "Same first letter keep their input order (stable):" << endl;
    cout << "apple before avocado, banana before blueberry, mango before melon" << endl;
    return 0;
}
```

Output

```
apple avocado banana blueberry cherry mango melon
Same first letter keep their input order (stable):
apple before avocado, banana before blueberry, mango before melon
```

2. Find median after sorting

```
#include <iostream>
#include <iomanip>
#include <vector>
using namespace std;

void insertionSort(vector<int> &arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

double median(vector<int> arr) {
    insertionSort(arr);
    int n = arr.size();
    if (n % 2 == 1) {
        return arr[n / 2];                      // odd: the middle one
    }
    return (arr[n / 2 - 1] + arr[n / 2]) / 2.0;  // even: average of the two middles
}

int main() {
    vector<int> odd = {7, 1, 5, 3, 9};
    vector<int> even = {8, 2, 7, 4, 3, 10};
    cout << fixed << setprecision(1);
    cout << "Median of {7, 1, 5, 3, 9}     = " << median(odd) << endl;
    cout << "Median of {8, 2, 7, 4, 3, 10} = " << median(even) << endl;
    return 0;
}
```

Output

```
Median of {7, 1, 5, 3, 9}     = 5.0
Median of {8, 2, 7, 4, 3, 10} = 5.5
```

Lesson 3: std::sort & Comparators

Quick recap: Using the STL std::sort • Custom comparators with lambdas • Sorting pairs and structs

Theory & Nuances

Using std::sort

Writing your own sort is for learning; in real code use sort from <algorithm>. It takes a range: sort(v.begin(), v.end()); sorts a whole vector, sort(arr, arr + n); sorts a C array of size n. It runs in O(n log n) - for n = 10^5 that is about 1.7 million steps instead of 10^10 for an O(n^2) sort. For descending order pass a comparator: sort(v.begin(), v.end(), greater<int>());. std::sort is NOT stable; use stable_sort with the same arguments when equal elements must keep their order.

Custom comparators with lambdas

A comparator is a function cmp(a, b) that returns true when a must come BEFORE b. A lambda writes it right where you need it: sort(v.begin(), v.end(), [](int a, int b) { return a > b; });. Rules: return a strict "less than" (use < or >, never <= or >=), and cmp(a, a) must be false - breaking this can crash std::sort. To break ties, compare the second key when the first keys are equal: if (a.size() != b.size()) return a.size() < b.size(); return a < b;. Writing [&] instead of [] lets the lambda use local variables, for example a frequency array.

Sorting pairs

pair<int, int> (from <utility>) holds two values, .first and .second. Make one with {3, 4} or make_pair(3, 4). By default sort orders pairs by first, and by second only when the firsts are equal. To sort by the second value, pass a comparator: [](const pair<string, int> &a, const pair<string, int> &b) { return a.second < b.second; }. Taking the parameters by const reference avoids copying.

Sorting structs

A struct groups related fields: struct Student { string name; int marks; };. sort does not know how to compare two Students, so you must always give a comparator that says which field to use, for example return a.marks > b.marks; for highest marks first. The same tie-breaking idea applies: compare marks first, and names only when the marks are equal.

Sorting as a first step

Many problems become easy once the data is sorted. Merge Intervals: sort intervals by start time, then each interval either overlaps the last merged one (extend its end) or starts a new one - one O(n) pass after an O(n log n) sort. Sort by frequency: count each value, then sort with a comparator that looks at the counts.

Key Points / Common Mistakes

- sort(v.begin(), v.end()) sorts the whole vector; sort(arr, arr + n) for a C array.
- A comparator returns true if a should come BEFORE b. Use < or >, never <= or >=.
- Descending: greater<int>() or the lambda [](int a, int b) { return a > b; }.
- Pairs sort by .first, then .second by default; structs need a comparator.
- std::sort is not stable - use stable_sort, or add a tie-breaker in the comparator.
- Use [&] when the lambda needs local variables (like a frequency array).

Classwork Problems

1. Sort a vector ascending / descending

```cpp
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

void printArray(const vector<int> &v) {
    for (int x : v) {
        cout << x << " ";
    }
    cout << endl;
}

int main() {
    vector<int> v = {42, 7, 19, 3, 25, 11};

    sort(v.begin(), v.end());                       // ascending
    cout << "Ascending          : ";
    printArray(v);

    sort(v.begin(), v.end(), greater<int>());       // descending
    cout << "Descending         : ";
    printArray(v);

    sort(v.begin(), v.end(), [](int a, int b) {     // a lambda comparator
        return a < b;
    });
    cout << "Ascending (lambda) : ";
    printArray(v);

    int arr[] = {9, 4, 6, 1};
    sort(arr, arr + 4);                             // a plain array
    cout << "Plain array        : ";
    for (int i = 0; i < 4; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
```

Output

```
Ascending          : 3 7 11 19 25 42
Descending         : 42 25 19 11 7 3
Ascending (lambda) : 3 7 11 19 25 42
Plain array        : 1 4 6 9
```

2. Sort vector of pairs by the second value

```
#include <iostream>
#include <algorithm>
#include <string>
#include <utility>
#include <vector>
using namespace std;

struct Student {
    string name;
    int marks;
};

int main() {
    vector<pair<string, int>> v = {{"Ravi", 72}, {"Asha", 85}, {"Kiran", 64}, {"Meena", 91}};

    sort(v.begin(), v.end());                       // default: by .first (name)
    cout << "By name : ";
    for (auto p : v) {
        cout << p.first << "(" << p.second << ") ";
    }
    cout << endl;

    sort(v.begin(), v.end(), [](const pair<string, int> &a, const pair<string, int> &b) {
        return a.second < b.second;                 // by .second (marks)
    });
    cout << "By marks: ";
    for (auto p : v) {
        cout << p.first << "(" << p.second << ") ";
    }
    cout << endl;

    // the same idea with a struct: highest marks first
    vector<Student> s = {{"Ravi", 72}, {"Asha", 85}, {"Kiran", 64}, {"Meena", 91}};
    sort(s.begin(), s.end(), [](const Student &a, const Student &b) {
        return a.marks > b.marks;
    });
    cout << "Struct, best first: ";
    for (auto st : s) {
        cout << st.name << "(" << st.marks << ") ";
    }
    cout << endl;
    return 0;
}
```

Output

```
By name : Asha(85) Kiran(64) Meena(91) Ravi(72)
By marks: Kiran(64) Ravi(72) Asha(85) Meena(91)
Struct, best first: Meena(91) Asha(85) Ravi(72) Kiran(64)
```

3. Sort strings by length

```
#include <iostream>
#include <algorithm>
#include <string>
#include <vector>
using namespace std;

bool shorterFirst(const string &a, const string &b) {
    if (a.size() != b.size()) {
        return a.size() < b.size();                // shorter word first
    }
    return a < b;                                  // same length: alphabetical
}

int main() {
    vector<string> words = {"banana", "kiwi", "apple", "fig", "cherry", "plum"};

    sort(words.begin(), words.end(), shorterFirst);
    for (const string &w : words) {
        cout << w << " (" << w.size() << ")" << endl;
    }
    return 0;
}
```

Output

```
fig (3)
kiwi (4)
plum (4)
apple (5)
banana (6)
cherry (6)
```

Homework Problems

1. LC56 Merge Intervals (sort by start time)

```
#include <iostream>
#include <algorithm>
#include <utility>
#include <vector>
using namespace std;

vector<pair<int, int>> merge(vector<pair<int, int>> intervals) {
    sort(intervals.begin(), intervals.end());       // by start time
    vector<pair<int, int>> merged;
    for (auto cur : intervals) {
        if (merged.empty() || cur.first > merged.back().second) {
            merged.push_back(cur);                  // no overlap: new interval
        } else {
            merged.back().second = max(merged.back().second, cur.second);  // overlap: extend
        }
    }
    return merged;
}

int main() {
    vector<pair<int, int>> intervals = {{1, 3}, {8, 10}, {2, 6}, {15, 18}, {9, 12}};

    vector<pair<int, int>> result = merge(intervals);
    cout << "Merged: ";
    for (auto p : result) {
        cout << "[" << p.first << ", " << p.second << "] ";
    }
    cout << endl;
    return 0;
}
```

Output

```
Merged: [1, 6] [8, 12] [15, 18]
```

2. Sort array elements by frequency

```
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {4, 6, 2, 6, 4, 4, 9, 2, 6, 6};
    int freq[10] = {0};                              // values are 0..9
    for (int x : v) {
        freq[x]++;
    }

    sort(v.begin(), v.end(), [&](int a, int b) {
        if (freq[a] != freq[b]) {
            return freq[a] > freq[b];                // more frequent first
        }
        return a < b;                                // tie: smaller value first
    });

    cout << "By frequency: ";
    for (int x : v) {
        cout << x << " ";
    }
    cout << endl;
    return 0;
}
```

Output

```
By frequency: 6 6 6 6 4 4 4 2 2 9
```